

REACT.JS · INTERVIEW PREPARATION

</>

5 React Interview Questions Every Fresher Should Know

React Interview Revision Guide for Freshers

Created by

Devika Jangid

5 QUESTIONS · FRESHER LEVEL · QUICK REVISION

WHAT'S INSIDE

- 1 Props vs State
- 2 useState Hook
- 3 useEffect Hook
- 4 Virtual DOM
- 5 Controlled vs Uncontrolled Components

Props vs State

In React, **props** and **state** are the two core mechanisms for managing and passing data — almost always the very first question a fresher is asked.

Props (Properties)

- **Read-only** data passed from a parent component to a child
- Cannot be modified by the component receiving them
- Used to configure or customize a component from outside
- Conceptually similar to function parameters

State

- Data managed **internally** within a component
- **Mutable** — changed using a setter function (or Hook)
- Updating it **triggers a re-render**
- Similar to a local variable with memory

KEY DIFFERENCES

ASPECT	PROPS	STATE
Mutability	Read-only / immutable	Mutable via setter / Hook
Ownership	Passed down from parent	Owned by the component itself
Typical use	Configuration & communication	Internal, dynamic data

WHEN TO USE EACH

Use **props** to pass data and callback functions down to child components. Use **state** for data that changes over time — based on user interaction, input, or lifecycle events — that the component itself owns.

SIMPLE EXAMPLE

```
function Profile({ name }) { // props
  const [likes, setLikes] = useState(0); // state
  return (
    <div>
      <h2>{name}</h2>
      <button onClick={() => setLikes(likes + 1)}>
        Like ({likes})
      </button>
    </div>
  );
}
```

COMMON INTERVIEW FOLLOW-UPS

1. Can a child component modify its parent's props?

No — props are read-only from the child's perspective. The child calls a function passed down as a prop, which updates the parent's own state.

2. Does changing state always cause a re-render?

Yes, calling the state setter schedules a re-render — even if you set it to the same value, React usually still re-renders unless it bails out via an internal check.

Interview Tip: Say confidently — "Props flow down, state lives inside." This single line shows you understand React's one-way data flow.

useState Hook

The `useState` Hook is the most fundamental React Hook — it lets functional components hold and update their own local state.

WHY IS IT USED?

Before Hooks, only class components could hold state. `useState` brings that same power to simple functional components, keeping code shorter and easier to read.

SYNTAX

```
const [state, setState] = useState(initialValue);
```

HOW STATE UPDATES WORK

- Calling the setter function **schedules a re-render** — it does not update the value instantly in the current render.
- React may **batch** multiple state updates together for better performance.
- Use the **functional updater** form when the new state depends on the previous state.

SIMPLE PRACTICAL EXAMPLE

```
function Counter() {  
  const [count, setCount] = useState(0);  
  const increment = () => setCount(prev => prev + 1);  
  return <button onClick={increment}>Count: {count}</button>;  
}
```

COMMON MISTAKES

- Mutating state directly instead of calling the setter function
- Expecting the updated value **immediately** after calling the setter
- Using stale state inside closures instead of the functional updater

COMMON INTERVIEW FOLLOW-UPS

1. Why does console.log right after setCount() show the old value?

State updates are asynchronous and only apply on the next render — the current closure still holds the old value.

2. What happens if you call useState inside a condition?

It breaks React's Hook call order between renders, which can cause bugs or errors — Hooks must always run in the same order on every render.

Interview Tip: Mention the functional updater form `setCount(prev => prev + 1)` when updating based on previous state — it shows you understand batching and stale closures.

useEffect Hook

`useEffect` lets you perform **side effects** — like data fetching, subscriptions, or manually updating the DOM — inside function components.

WHY DO WE USE IT?

It replaces the class-component lifecycle methods `componentDidMount`, `componentDidUpdate`, and `componentWillUnmount` with one unified, declarative API.

DEPENDENCY ARRAY

FORM	BEHAVIOUR
No array	Effect runs after every render
<code>[]</code> empty array	Effect runs once , right after the first render
<code>[dep1, dep2]</code>	Effect re-runs only when a listed value changes

CLEANUP FUNCTION

Returning a function from `useEffect` runs it **before** the component unmounts, or before the effect re-runs — used to remove listeners, clear timers, or cancel subscriptions.

PRACTICAL EXAMPLE

```
useEffect(() => {
  const timer = setInterval(() => console.log('tick'), 1000);
  return () => clearInterval(timer); // cleanup
}, []); // runs once on mount
```

COMMON MISTAKES

Forgetting the dependency array (causing effects to run on every render); missing the cleanup function for subscriptions/timers; and updating state inside an effect without a proper dependency array, causing infinite loops.

COMMON INTERVIEW FOLLOW-UPS

1. What happens if you forget the dependency array?

The effect runs after every render, which can cause performance issues — or infinite loops if it also updates state.

2. When does the cleanup function actually run?

Before the component unmounts, and also before the effect re-runs on a dependency change — React cleans up the previous effect first.

Interview Tip: Always explain all three variants — no array, empty array, and array with values — interviewers often test whether you understand the difference in one go.

Virtual DOM

The **Virtual DOM (VDOM)** is a core reason React apps feel fast — and interviewers love asking freshers to explain it clearly and simply.

Real DOM

- The actual browser structure rendered on screen
- Updating it directly is **slow & expensive**
- Every change can trigger layout, reflow & repaint

Virtual DOM

- A lightweight in-memory JS object copy of the real DOM
- Cheap to create and compare (diff)
- React patches only what actually changed

RECONCILIATION

The process React uses to compare the new Virtual DOM tree against the previous one — its **diffing algorithm** — and calculate the minimal set of real DOM changes required.

WHY REACT USES IT

Directly manipulating the real DOM for every state change is slow. The Virtual DOM lets React **batch and minimize** actual DOM operations, keeping UI updates fast and predictable.

SIMPLE VISUAL EXPLANATION



PRACTICAL EXAMPLE

When `count` changes in `setCount(count + 1)`, React creates a new Virtual DOM tree, diffs it against the last one, and updates only the text node showing the count — not the entire page.

COMMON INTERVIEW FOLLOW-UPS

1. Does the Virtual DOM make React faster than plain JavaScript?

Not always — it makes DOM updates more efficient and predictable at scale. For tiny, isolated changes, direct DOM manipulation can sometimes be faster.

2. Is the Virtual DOM unique to React?

No — the concept is used by other frameworks too, but React popularized it as a core part of its rendering model.

Interview Tip: Use the exact term "reconciliation" — it signals you know React's diffing algorithm by name, not just the general idea.

Controlled vs Uncontrolled Components

This question tests how well you understand **form handling** in React — a very common real-world scenario.

Controlled Component

- Input's value is driven by **React state**
- Every keystroke updates state via **onChange**
- React is the **single source of truth**

Uncontrolled Component

- Input manages its own value inside the **DOM**
- Value is read via a **ref** when needed
- The DOM is the source of truth

KEY DIFFERENCES

ASPECT	CONTROLLED	UNCONTROLLED
Source of truth	React state	The DOM itself
Value access	Always available in state	Read on demand via ref
Validation	Easy, instant	Harder, done on submit

FORM EXAMPLE

```
// Controlled
<input value={name} onChange={e => setName(e.target.value)} />

// Uncontrolled
<input ref={inputRef} defaultValue="Guest" />
// read later: inputRef.current.value
```

WHEN TO USE EACH

Use **controlled** components for most forms needing validation or dynamic UI updates. Use **uncontrolled** components for simple forms, file inputs, or integrating with non-React code.

COMMON INTERVIEW CONFUSION

Freshers often think "uncontrolled" means React has no involvement at all — it still renders the input, it just doesn't manage its *value* through state.

COMMON INTERVIEW FOLLOW-UPS

1. Why can't file inputs be fully controlled?

Browsers restrict setting a file input's value programmatically for security, so it's typically handled as an uncontrolled component via a ref.

2. Which approach does React recommend?

Controlled components for most cases, since they keep the UI predictable — but uncontrolled ones are fine for simple, one-off inputs.

Interview Tip: Say: "Controlled means React owns the value; uncontrolled means the DOM owns it." A short, clear line like this impresses interviewers.